

Stack - LIFO

last in first out

6	2	9	8	1	1
---	---	---	---	---	---

↑
s.top

Stack - empty

if s.top == 0
return true

return false

Push (s, x)

if (s.top < size) {

s[s.top+1] = x

s.top ++

3

pop(s)

if s.top == 0

error "underflow")

x = s.top

s.top --

return x

Queue - FIFO

first in first out

Q.head, Q.tail - attributes

Enqu(Q, x)

if (Q.numelements == size)

return overflow

if (Q.tail == size)

Q.tail = 1

else

Q.tail ++

Q.numelements ++

Q[Q.tail] = x

Dequeue (Q)

```
if (Q.numElements == 0)
    return underflow
```

```
x = Q.head
```

```
if (Q.head == size)
```

```
    Q.head = 1
```

```
else
```

```
    Q.head++
```

```
Q.numElements--
```

```
return x
```

Linked Lists

Singly linked

insert (L, x)

```
x.next = L.head
```

```
L.head = x
```

Search (L, k)

```
curr = L.head
```

```
while (curr != NIL &&  
        curr.key != k)
```

```
    curr = curr.next
```

```
return curr
```

Delete (L, x)

curr = L.head

prev = NIL

while (curr != NIL &&

curr != x)

prev = curr

curr = curr.next

if (curr == NIL) return

if (prev != NIL)

prev → next = x.next

else

L.head = x.next

delete x

Doubly linked lists

delete (L, x)

if (x.prev != NIL)

x.prev.next = x.next

else

L.head = x.next

if (x.next != NIL)

x.next.prev = x.prev

insert (L, x)

x.next = L.head

if (L.head)

L.head.prev = x

x.prev = NIL

L.head = x